Tecnicatura Superior en Desarrollo de Software

Módulo 08 — Infrastructure: `DbContext` y Repositorios

Actividad Práctica — CU-CLI-001: Registrar Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

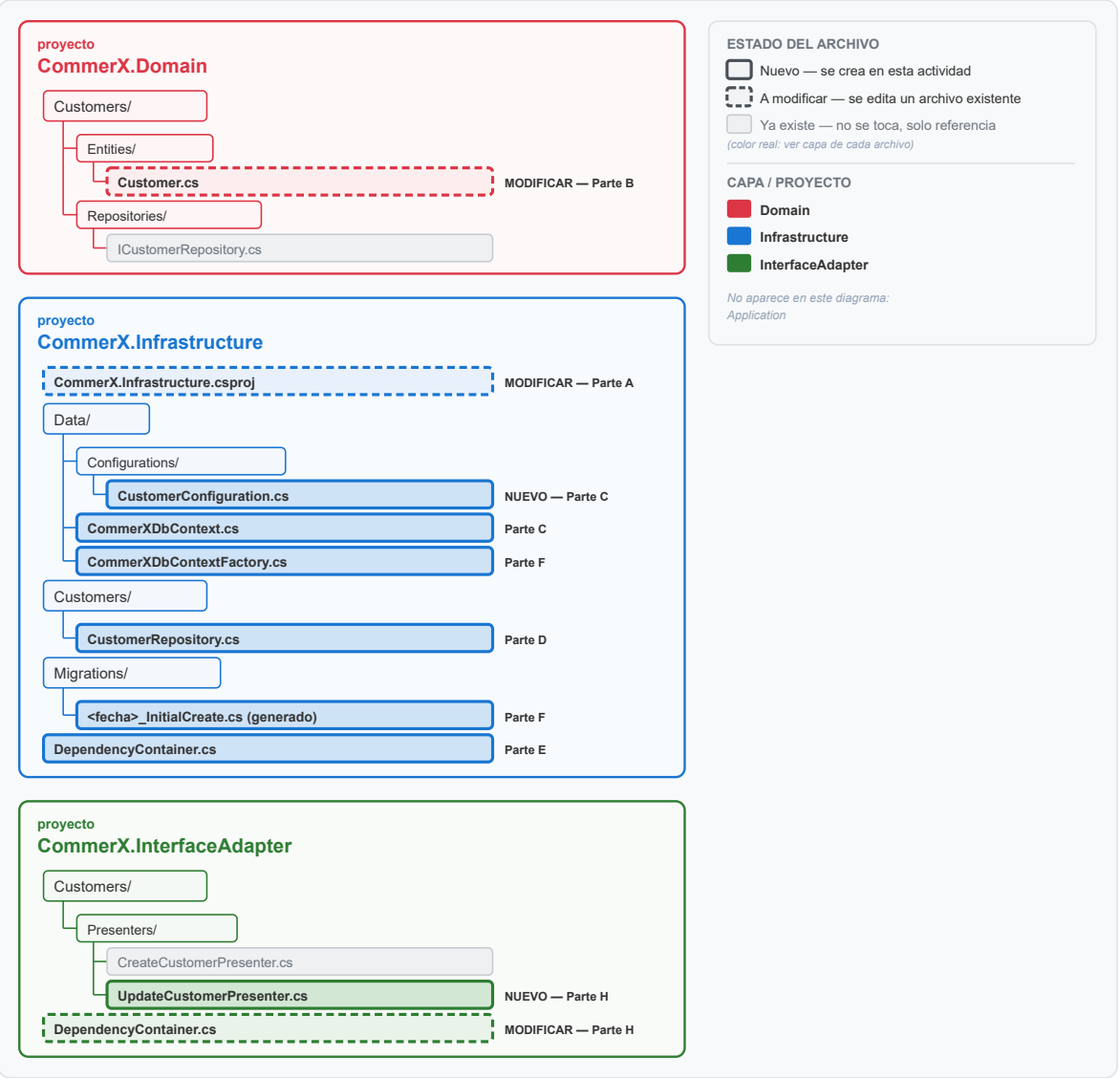
Contexto de la actividad

En los módulos anteriores construiste la lógica de negocio (Domain), los casos de uso y los puertos (Application) y los presenters (InterfaceAdapter). Pero esa lógica vive en memoria: los clientes que registrás se pierden cuando cerrás la aplicación.

En esta actividad Infrastructure toca el mundo real: vas a configurar un `DbContext` de Entity Framework Core, a crear el repositorio concreto `CustomerRepository` (la clase que cumple el contrato `ICustomerRepository` que ya definiste en Domain) y a registrar todo en el contenedor de inyección de dependencias. Al terminar, los clientes se guardan en una base de datos SQL Server.

Esta actividad **no construye el host de escritorio** (WPF con Blazor Hybrid), que es quien va a mostrar las pantallas. Infrastructure queda lista para que el host la use con una sola línea, `AddInfrastructure(...)`, y mientras tanto comprobás que todo funciona mirando la base de datos que crea la migración (Parte F).

Archivos involucrados



Cada archivo lleva el color del proyecto al que pertenece. Los de borde sólido son **NUEVOS** (los creás de cero), los de borde discontinuo se **MODIFICAN** (ya existen de módulos anteriores) y los grises ya existen y no se tocan: solo sirven de referencia. La letra de cada archivo es la de la Parte donde lo trabajás.

Parte A — Referencias del proyecto: instalar Entity Framework Core

Ubicación

`src/Infrastructure/CommerX.Infrastructure.csproj` (se modifica con los pasos de abajo)

Qué es una referencia y por qué hay que agregarla

Un proyecto de C# solo puede usar el código que **declaró explícitamente**: si en un archivo escribís `using Microsoft.EntityFrameworkCore;` y el proyecto no tiene esa librería declarada, el compilador responde *"el espacio de nombres no existe"*. No hay nada automático: cada dependencia está escrita en el archivo `.csproj` del proyecto. Hay dos tipos de referencia:

Tipo	En el .csproj	Qué trae	Ejemplo en CommerX
Referencia a proyecto	`<ProjectReference>`	El código de otro proyecto de *tu misma solución*	Infrastructure referencia a Domain para poder usar `Customer` e `ICustomerRepository`
Paquete NuGet	`<PackageReference>`	Una librería ya compilada que publicó otra persona u organización (se descarga de nuget.org)	Infrastructure referencia a Entity Framework Core para hablar con SQL Server

NuGet es el "almacén" de librerías de .NET. Un *paquete* es una librería con un nombre y una versión; al agregarlo, NuGet lo descarga una vez a una carpeta de tu máquina (no a tu repositorio) y el compilador lo usa desde ahí. Por eso la carpeta del proyecto no se llena de archivos ajenos: solo queda escrita una línea en el `.csproj` que dice "este proyecto necesita el paquete X, versión Y".

Qué vamos a agregar, y para qué

Qué	Para qué sirve	Qué te permite escribir
`Microsoft.EntityFrameworkCore.SqlServer` (paquete)	Es Entity Framework Core más el *proveedor* de SQL Server: la pieza que traduce tus consultas en C# al SQL que entiende ese motor.	`DbContext`, `DbSet`, `ModelBuilder`, `UseSqlServer()`, `FirstOrDefaultAsync()`…
`Microsoft.EntityFrameworkCore.Design` (paquete)	Lo que necesitan las herramientas de migraciones para analizar tu proyecto y generar la base de datos (Parte F).	Nada en tu código: solo lo usa `dotnet ef`. Por eso se marca `PrivateAssets="all"`: no se propaga a los proyectos que referencien a Infrastructure.
`dotnet-ef` (herramienta global)	El programa de línea de comandos `dotnet ef`. No es una referencia de ningún proyecto: se instala una sola vez en tu máquina.	Los comandos `dotnet ef migrations add` y `dotnet ef database update`.

Por qué solo en Infrastructure

La regla de dependencias de Clean Architecture se materializa en estas referencias: una capa solo puede referenciar a las que tiene *más adentro*. Así quedan hoy los proyectos:

Proyecto	Referencia a proyectos	Paquetes de terceros
`CommerX.Domain`	ninguno	ninguno
`CommerX.Application`	Domain	abstracciones de inyección de dependencias
`CommerX.InterfaceAdapter`	Application	abstracciones de inyección de dependencias
`CommerX.Infrastructure`	Domain	**Entity Framework Core (esta parte)**

> **Nunca agregues Entity Framework Core a Domain ni a Application.** Si lo hicieras, el negocio dependería de una tecnología de persistencia y no podrías cambiarla sin tocarlo. Si el compilador te sugiere un `using Microsoft.EntityFrameworkCore;` en esas capas, es una señal de que el código está en el proyecto equivocado.

Infrastructure **ya referencia a Domain** (es donde vive `ICustomerRepository`): no hay que agregar ninguna referencia entre proyectos. Lo que falta son los paquetes. Podés instalarlos de dos maneras equivalentes; elegí una.

Opción 1 — Desde la línea de comandos

Abrí una terminal en la carpeta raíz de la solución (la que contiene `CommerX.slnx`) y ejecutá, uno por uno:

```
dotnet add src/Infrastructure/CommerX.Infrastructure.csproj package Microsoft.EntityFrameworkCore.SqlServer --version 9.0.20
dotnet add src/Infrastructure/CommerX.Infrastructure.csproj package Microsoft.EntityFrameworkCore.Design --version 9.0.20
dotnet tool install --global dotnet-ef
```

1. **Comando 1:** `dotnet add ... package` le dice a .NET "agregá este paquete a este proyecto". Descarga el paquete (y las librerías de las que depende) y escribe la línea `<PackageReference>` en el `.csproj`. El nombre después de `package` es el del paquete en nuget.org y `--version` fija la versión.
2. **Comando 2:** lo mismo con el paquete de diseño.
3. **Comando 3:** `dotnet tool install --global` instala una herramienta en tu máquina (no en el proyecto). Si ya la tenés, el comando avisa y no pasa nada.

Opción 2 — Desde Visual Studio

1. En el **Explorador de soluciones**, hacé clic derecho sobre el proyecto `CommerX.Infrastructure` (no sobre la solución) y elegí **Administrar paquetes NuGet…**
2. Pasá a la pestaña **Examinar** y escribí en el buscador `Microsoft.EntityFrameworkCore.SqlServer`.
3. Seleccioná el paquete en la lista. En el panel de la derecha verificá que **Proyecto** sea `CommerX.Infrastructure` y desplegá **Versión**: elegí `9.0.20` (más abajo se explica por qué no la última).
4. Presioná **Instalar** y aceptá la ventana de vista previa de cambios.
5. Repetí los pasos 2 a 4 con `Microsoft.EntityFrameworkCore.Design`.
6. La herramienta `dotnet-ef` no se instala desde esta ventana: abrí **Ver** → **Terminal** y ejecutá el comando 3 de la opción 1.

> **Por qué la versión 9.0.20 y no la última:** la solución apunta a `net9.0` y cada versión mayor de EF Core exige la versión mayor equivalente de .NET. Si instalás EF Core 10 en un proyecto `net9.0`, NuGet rechaza el paquete con un error de compatibilidad (`NU1202`). Fijar la versión exacta además hace que todo el curso compile lo mismo.

Qué cambió en el .csproj

Cualquiera de las dos opciones modifica el mismo archivo. Es el único cambio: abrilo (doble clic sobre el proyecto en Visual Studio, o en cualquier editor de texto) y compará.

Antes — solo la referencia a Domain:

```
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <ProjectReference Include="..\Domain\CommerX.Domain.csproj" />
  </ItemGroup>

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Después — se agrega un grupo con los dos paquetes:

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
2
3     <ItemGroup>
4       <ProjectReference Include="..\Domain\CommerX.Domain.csproj" />
5     </ItemGroup>
6
7     <ItemGroup>
8       <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.20">
9         <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
  IncludeAssets>
10        <PrivateAssets>all</PrivateAssets>
11      </PackageReference>
12      <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.2
  0" />
13    </ItemGroup>
14
15    <PropertyGroup>
16      <TargetFramework>net9.0</TargetFramework>
17      <ImplicitUsings>enable</ImplicitUsings>
18      <Nullable>enable</Nullable>
19    </PropertyGroup>
20
21  </Project>
```

Cómo leerlo:
Línea 8: `PackageReference` es la declaración del paquete: `Include` es su nombre y `Version` la versión exacta.

Línea 10: `PrivateAssets` con valor `all` significa que el paquete de diseño sirve para este proyecto pero no se contagia a quien lo referencie.

Línea 12: el paquete de SQL Server, sin `PrivateAssets`: lo necesitan tanto Infrastructure como, indirectamente, cualquier proyecto que ejecute la aplicación.

Cómo comprobar que quedó bien

Estos dos comandos listan los paquetes del proyecto y compilan:

```
1  dotnet list src/Infrastructure/CommerX.Infrastructure.csproj package
2  dotnet build src/Infrastructure/CommerX.Infrastructure.csproj
```

> **Resultado esperado:** `dotnet list ... package` muestra `Microsoft.EntityFrameworkCore.Design` y `Microsoft.EntityFrameworkCore.SqlServer` en `9.0.20`, y `dotnet build` termina con *0 Advertencias, 0 Errores*. La primera vez tarda más porque NuGet está descargando los paquetes (esa descarga se llama *restauración*); las siguientes compilaciones ya los encuentran en tu máquina.

Parte B — Modificar `Customer.cs`: el constructor que necesita EF Core

Ubicación

`src/Domain/Customers/Entities/Customer.cs`

Descripción

Cuando EF Core lee una fila de la base tiene que *reconstruir* un `Customer`, y para eso necesita un constructor que pueda llamar. El único que tiene hoy `Customer` recibe los siete Value Objects por parámetro, y EF Core no puede completar parámetros con Value Objects: los mapea como *tipos propios* (`OwnsOne`, Parte C) y solo sabe enlazar por constructor las propiedades simples. Si te salteás este paso, la primera consulta falla con:

> `InvalidOperationException: No suitable constructor was found for entity type 'Customer'.`
> `Note that only mapped properties can be bound to constructor parameters. Navigations to related entities, including references to owned types, cannot be bound.`

La solución es agregar un constructor **privado y sin parámetros**, junto a los que ya existen:

```
    // constructor sin parámetros — solo lo usa Entity Framework Core para reconstruir el Customer desde la base
    // EF no puede enlazar Value Objects por constructor: crea la instancia vacía y completa las propiedades por reflexión
    // al ser privado, el código de negocio sigue obligado a pasar por Create()
    private Customer()
    {
        FirstName = null!;
        LastName = null!;
```

```
8           Document = null!;
9           Email = null!;
10          Phone = null!;
11          Address = null!;
12          BirthDate = null!;
13      }
```

Qué está pasando:
Línea 4: el constructor es `private`: el código de negocio sigue obligado a pasar por `Customer.Create()` (y por la validación de los Value Objects). Solo Entity Framework Core, que usa reflexión, puede invocarlo.

Líneas 6 a 12: como las propiedades no son nulables, el compilador exige inicializarlas; `null!` le dice "ya sé que es nulo acá, EF Core lo completa enseguida". Nunca queda un `Customer` incompleto circulando por el dominio.

Domain sigue sin conocer a EF Core: no hay ninguna referencia nueva, solo un constructor privado. La dirección de dependencias no cambió.

> **Para profundizar:** el anexo `anexo-08a-entity-framework-fluent-api.html` explica qué espera EF Core de una entidad (constructores, propiedades de solo lectura, tipos propios).

Parte C — Configurar el mapeo: `CustomerConfiguration.cs` y `CommerXDbContext.cs`

Ubicación

`src/Infrastructure/Data/Configurations/CustomerConfiguration.cs` y `src/Infrastructure/Data/CommerXDbContext.cs` (los dos nuevos)

Descripción

El `CommerXDbContext` es el puente entre tu modelo de dominio (las entidades y Value Objects en C#) y las tablas de SQL Server. Define qué entidades se guardan (`DbSet`) y cómo se mapea cada una. Para que no se convierta en un método larguísimo cuando haya más entidades, el *cómo* se mapea cada entidad va en **su propia clase de configuración**, y el `DbContext` solo las enumera. Son dos archivos: primero la configuración de `Customer`, después el `DbContext` que la usa.

Paso 1 — La configuración de Customer

Una clase de configuración implementa `IEntityTypeConfiguration<T>`, la interfaz de EF Core pensada para esto, y su único método recibe un *constructor* (`EntityTypeBuilder`) de esa entidad. Creá una carpeta `Configurations` dentro de `Data` y en ella el archivo:

```
using CommerX.Domain.Customers.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace CommerX.Infrastructure.Data.Configurations;

// toda la configuración de Fluent API del Customer vive acá, no en el DbContext
public sealed class CustomerConfiguration : IEntityTypeConfiguration<Customer>
{
    public void Configure(EntityTypeBuilder<Customer> customer)
    {
        // tabla y clave primaria (el índice único del Id ya lo crea la clave primaria)
        customer.ToTable("Customers");
        customer.HasKey(c => c.Id);

        // auditoría de BaseEntity — CreatedAt solo tiene getter y por convención EF Core no mapea
        // propiedades de solo lectura: sin esta línea se regeneraría con la hora actual al cargar el cliente
        customer.Property(c => c.CreatedAt).IsRequired();
        customer.Property(c => c.UpdatedAt).IsRequired();

        // cada Value Object vive como columna dentro de Customers (Owned Type)
        // y su propiedad Value se configura con la longitud que valida el propio Value Object
        customer.OwnsOne(c => c.FirstName, vo =>
            vo.Property(x => x.Value).HasMaxLength(100).IsRequired());

        customer.OwnsOne(c => c.LastName, vo =>
            vo.Property(x => x.Value).HasMaxLength(100).IsRequired());

        // el documento y el email son únicos: el índice se declara dentro del Owned Type
        customer.OwnsOne(c => c.Document, vo =>
        {
            vo.Property(x => x.Value).HasMaxLength(8).IsRequired();
            vo.HasIndex(x => x.Value).IsUnique();
        });

        customer.OwnsOne(c => c.Email, vo =>
        {
            vo.Property(x => x.Value).HasMaxLength(254).IsRequired();
            vo.HasIndex(x => x.Value).IsUnique();
```

```
40          });
41
42          customer.OwnsOne(c => c.Phone, vo =>
43              vo.Property(x => x.Value).HasMaxLength(20).IsRequired());
44
45          customer.OwnsOne(c => c.Address, vo =>
46              vo.Property(x => x.Value).HasMaxLength(200).IsRequired());
47
48          customer.OwnsOne(c => c.BirthDate, vo =>
49              vo.Property(x => x.Value).IsRequired());
50
51          // un Customer siempre tiene sus siete Value Objects: ninguna columna queda nul
    lable
52          customer.Navigation(c => c.FirstName).IsRequired();
53          customer.Navigation(c => c.LastName).IsRequired();
54          customer.Navigation(c => c.Document).IsRequired();
55          customer.Navigation(c => c.Email).IsRequired();
56          customer.Navigation(c => c.Phone).IsRequired();
57          customer.Navigation(c => c.Address).IsRequired();
58          customer.Navigation(c => c.BirthDate).IsRequired();
59      }
60  }
```

Qué está pasando:
Líneas 8 y 10: la clase implementa la interfaz de configuración para `Customer`. El parámetro `customer` es el constructor sobre el que encadenás todo el mapeo.

Líneas 13 y 14: la tabla se llama `Customers` y su clave primaria es `Id`. La clave primaria ya crea un índice único, por eso no hace falta otro sobre `Id`.

Líneas 18 y 19: mapean `CreatedAt` y `UpdatedAt` de `BaseEntity`. `CreatedAt` solo tiene `get` y EF Core **no mapea por convención las propiedades de solo lectura**: sin la línea 18, la fecha de alta no se guardaría y cada vez que cargues un cliente aparecería con la hora de ese momento, sin ningún error que te avise.

Líneas 23 a 48: cada Value Object vive *dentro* de la tabla `Customers` como una columna (`FirstName_Value`, `Document_Value`, etc.): eso es `OwnsOne`. En cada uno se configura la propiedad `Value` con la misma longitud máxima que valida el propio Value Object. Ojo: `Property(x => x.Value)` no es solo un detalle de longitud. `Value` es de solo lectura y EF Core no mapea esas propiedades por convención; nombrándola la incorporás al modelo, y recién así EF puede pasarla al constructor privado del Value Object al leer la fila. Con un `OwnsOne` pelado falla con *"No suitable constructor was found for entity type 'Address'"*.

Líneas 33 y 39: `Document` y `Email` son únicos (regla de unicidad de CU-CLI-001).

El índice se declara *dentro* del `OwnsOne`, sobre `Value`: un índice sobre el Value Object completo no existe, la columna es la que lleva el índice.

Líneas 52 a 58: un cliente siempre tiene sus siete Value Objects, así que ninguna de esas columnas puede quedar nula en la base.

Paso 2 — El DbContext

Ahora el `DbContext`, que queda corto: declara la colección que representa la tabla y le indica a EF qué configuraciones aplicar.

```
1  using CommerX.Domain.Customers.Entities;
2  using CommerX.Infrastructure.Data.Configurations;
3  using Microsoft.EntityFrameworkCore;
4
5  namespace CommerX.Infrastructure.Data;
6
7  public sealed class CommerXDbContext : DbContext
8  {
9      public CommerXDbContext(DbContextOptions<CommerXDbContext> options)
10         : base(options)
11     {
12     }
13
14     // Set<T>() evita la advertencia de nulabilidad: EF Core resuelve el DbSet por su c
   uenta
15     public DbSet<Customer> Customers => Set<Customer>();
16
17     protected override void OnModelCreating(ModelBuilder modelBuilder)
18     {
19         // el DbContext solo enumera las configuraciones; el detalle de cada entidad es
   tá en su clase
20         modelBuilder.ApplyConfiguration(new CustomerConfiguration());
21     }
22 }
```

Qué está pasando:
Líneas 9 a 12: el contexto recibe sus opciones (qué proveedor usar, la cadena de conexión) por constructor. No las decide él: se las entrega el contenedor de dependencias (Parte E).

Línea 15: `Customers` es la colección que representa la tabla. `Set<Customer>()` evita la advertencia de nulabilidad de las propiedades `{ get; set; }` sin inicializar.

Línea 20: `ApplyConfiguration` registra la configuración de `Customer` de forma

explícita: el `DbContext` deja escrito qué configuraciones usa. Cuando agregues una entidad nueva, sumás su `DbSet` y una línea `ApplyConfiguration`.

> **Por qué una clase por entidad:** el `DbContext` no crece con cada entidad, cada mapeo se encuentra y se prueba en su propio archivo, y las configuraciones no dependen del motor de base de datos: si algún día cambiás de motor, estas clases se reutilizan sin tocarlas. (EF Core también puede descubrirlas solo con `ApplyConfigurationsFromAssembly`; acá se usa la forma explícita para que veas qué se aplica.)

> **Todo esto en detalle:** `anexo-08a-entity-framework-fluent-api.html` explica línea por línea cada método de Fluent API (propiedades, tipos propios, índices, relaciones) y cómo separar la configuración por entidad, con más ejemplos.

Parte D — Crear `CustomerRepository.cs`

Ubicación

`src/Infrastructure/Customers/CustomerRepository.cs`

Descripción

El `CustomerRepository` implementa `ICustomerRepository`, la interfaz que definiste en Domain (`src/Domain/Customers/Repositories/`) en el Módulo 04. Tiene que cumplir ese contrato **tal cual está**: mismos nombres, mismos parámetros y mismos tipos de retorno. Los casos de uso ya lo llaman así y no se tocan.

```
1  using CommerX.Domain.Customers.Entities;
2  using CommerX.Domain.Customers.Repositories;
3  using CommerX.Infrastructure.Data;
4  using Microsoft.EntityFrameworkCore;
5
6  namespace CommerX.Infrastructure.Customers;
7
8  public sealed class CustomerRepository : ICustomerRepository
9  {
10     private readonly CommerXDbContext _db;
11
12     public CustomerRepository(CommerXDbContext db)
13     {
14         _db = db;
15     }
```

```
16
17      // CU-CLI-001 — se compara contra la propiedad Value del Value Object: EF la traduc
    e a SQL
18      public async Task<Customer?> FindByDocumentAsync(string document)
19      {
20          return await _db.Customers
21              .FirstOrDefaultAsync(c => c.Document.Value == document);
22      }
23
24      public async Task<Customer?> FindByEmailAsync(string email)
25      {
26          return await _db.Customers
27              .FirstOrDefaultAsync(c => c.Email.Value == email);
28      }
29
30      public async Task AddAsync(Customer customer)
31      {
32          await _db.Customers.AddAsync(customer);
33          await _db.SaveChangesAsync();
34      }
35
36      // CU-CLI-002
37      public async Task<Customer?> GetByIdAsync(Guid id)
38      {
39          return await _db.Customers
40              .FirstOrDefaultAsync(c => c.Id == id);
41      }
42
43      public async Task UpdateAsync(Customer customer)
44      {
45          _db.Customers.Update(customer);
46          await _db.SaveChangesAsync();
47      }
48  }
```

Qué está pasando:
Líneas 18 a 21: la búsqueda por documento compara contra `c.Document.Value` (el `string`), no contra el Value Object entero. EF Core traduce esa expresión lambda a un `SELECT ... WHERE Document_Value = @p`. Lo mismo pasa con el email (línea 27).

Retorno `Customer?`: si no hay ninguna fila, `FirstOrDefaultAsync` devuelve `null` y es el caso de uso quien decide qué hacer (por ejemplo, avisar que el cliente no existe).

Líneas 32 y 33: `AddAsync` solo *marca* al cliente como nuevo en memoria; recién `SaveChangesAsync` ejecuta el `INSERT`.

Línea 37: `GetByIdAsync` es el método que usa CU-CLI-002 para localizar al cliente que se va a actualizar.

Líneas 45 y 46: `UpdateAsync` deja registrados los cambios del cliente y los persiste con `SaveChangesAsync` (un `UPDATE` de las columnas modificadas).

> **El repositorio solo persiste:** no valida, no decide reglas de negocio ni formatea mensajes. Todo eso ya ocurrió en Domain y Application antes de llegar acá.

Parte E — Crear `DependencyContainer.cs` de Infrastructure

Ubicación

`src/Infrastructure/DependencyContainer.cs`

Descripción

Igual que Application e InterfaceAdapter, Infrastructure tiene su propio `DependencyContainer` con un método de extensión que registra todo lo suyo. Así el host no necesita conocer ni el `DbContext` ni el repositorio: llama a un solo método.

```
1  using CommerX.Domain.Customers.Repositories;
2  using CommerX.Infrastructure.Customers;
3  using CommerX.Infrastructure.Data;
4  using Microsoft.EntityFrameworkCore;
5
6  // namespace truco — extiende IServiceCollection sin agregar using en Program.cs
7  namespace Microsoft.Extensions.DependencyInjection;
8
9  public static class DependencyContainer
10 {
11     public static IServiceCollection AddInfrastructure(
12         this IServiceCollection services,
13         string connectionString)
14     {
15         // DbContext con SQL Server — la cadena de conexión la entrega el host, no vive en el código
16         services.AddDbContext<CommerXDbContext>(options =>
17             options.UseSqlServer(connectionString));
18
19         // Scoped: el repositorio comparte el DbContext del scope en el que se resuelve
20         services.AddScoped<ICustomerRepository, CustomerRepository>();
```

```
21
22          return services;
23      }
24  }
```

Qué está pasando:
Línea 7: el mismo "namespace truco" de las otras capas: el método extiende `IServicCollection` sin obligar al host a agregar un `using`.

Líneas 16 a 17: `AddDbContext` registra el contexto con el proveedor de SQL Server. La cadena de conexión llega **por parámetro**: Infrastructure no la escribe en el código, la entrega quien arma la aplicación (el host, desde su configuración).

Línea 20: `AddScoped` asocia la interfaz de Domain con la clase concreta. Cada vez que alguien pida un `ICustomerRepository` en un mismo *scope* recibe la misma instancia, que comparte el `DbContext` de ese scope (por defecto `AddDbContext` también es Scoped).

Cuando exista el host, la composición completa será una sola línea por capa. La cadena de conexión de desarrollo apunta a SQL Server LocalDB, que se instala con Visual Studio:

```
1  services.AddInfrastructure(
2      "Server=(localdb)\\MSSQLLocalDB;Database=CommerX;Trusted_Connection=True;");
```

> **Si tu servidor no es LocalDB** (por ejemplo SQL Server Developer con un certificado de desarrollo), el controlador cifra la conexión por defecto y puede rechazar el certificado: en desarrollo agregá `TrustServerCertificate=True;` a la cadena.

Parte F — Crear la base de datos con una migración

Ubicación

`src/Infrastructure/Data/CommerXDbContextFactory.cs` (nuevo) y `src/Infrastructure/Migrations/` (la genera `dotnet ef`)

Descripción

Una **migración** es un archivo C# versionado que describe cómo pasar la base de datos de un estado al siguiente. Entity Framework la calcula comparando tu modelo (`CommerXDbContext`) con el estado anterior, y la herramienta `dotnet ef` la aplica sobre SQL Server. Nunca editás la base a mano.

Para generar migraciones, `dotnet ef` necesita construir tu `DbContext` por su cuenta. Como todavía no hay un host que lo configure, le enseñamos cómo con una *fábrica de tiempo de diseño*:

```
1  using Microsoft.EntityFrameworkCore;
2  using Microsoft.EntityFrameworkCore.Design;
3
4  namespace CommerX.Infrastructure.Data;
5
6  // solo la usan las herramientas de EF (dotnet ef migrations / database update) en tiem
   po de diseño
7  // permite generar migraciones sin un proyecto host: el host real recibe su cadena de c
   onexión por DI
8  public sealed class CommerXDbContextFactory : IDesignTimeDbContextFactory<CommerXDbCont
   ext>
9  {
10     private const string DesignTimeConnectionString =
11         "Server=(localdb)\\MSSQLLocalDB;Database=CommerX;Trusted_Connection=True;";
12
13     public CommerXDbContext CreateDbContext(string[] args)
14     {
15         var options = new DbContextOptionsBuilder<CommerXDbContext>()
16             .UseSqlServer(DesignTimeConnectionString)
17             .Options;
18
19         return new CommerXDbContext(options);
20     }
21 }
```

Qué está pasando:
Línea 8: `IDesignTimeDbContextFactory` es la interfaz que busca `dotnet ef`. Esta clase **solo la usan las herramientas**, nunca la aplicación en ejecución (esa recibe su contexto del contenedor de dependencias).

Línea 10: la cadena de conexión de desarrollo, apuntando a LocalDB. Es la única vez que una cadena aparece escrita en el código, y solo para el diseño de la base.

Ahora generá y aplicá la migración (los comandos se ejecutan dentro de la carpeta de Infrastructure):

```
cd src/Infrastructure
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Qué está pasando:
Línea 2: crea la carpeta `Migrations/` con tres archivos: la migración (`<fecha>_InitialCreate.cs`, con los métodos `Up` y `Down`), su archivo `.Designer.cs` y `CommerXDbContextModelSnapshot.cs`, la "foto" del modelo que EF usa para calcular la próxima migración. Son archivos generados: se guardan en Git pero no se editan a mano.

Línea 3: crea la base de datos `CommerX` en LocalDB (si no existe) y ejecuta la migración.

Cómo comprobar el resultado

Abrí la base `CommerX` en Visual Studio (Explorador de objetos de SQL Server) o en Azure Data Studio. La tabla `Customers` tiene que quedar así:

Columna	Tipo en SQL Server	¿Acepta nulos?
`Id`	uniqueidentifier (clave primaria)	No
`FirstName_Value`	nvarchar(100)	No
`LastName_Value`	nvarchar(100)	No
`Document_Value`	nvarchar(8) — índice único	No
`Email_Value`	nvarchar(254) — índice único	No
`Phone_Value`	nvarchar(20)	No
`Address_Value`	nvarchar(200)	No
`BirthDate_Value`	date	No
`CreatedAt`	datetime2	No
`UpdatedAt`	datetime2	No

Tres índices: `PK_Customers` (la clave primaria), `IX_Customers_Document_Value` e `IX_Customers_Email_Value` (los dos únicos). Si intentás insertar a mano dos filas con el mismo documento, la base rechaza la segunda.

Parte G — Flujo completo: CU-CLI-001 de punta a punta

Con Infrastructure lista, mirá qué pasa cuando un usuario registra un cliente (el recorrido es el mismo cuando el host exista; hoy lo recorre el código de un test):

1. **El usuario hace clic en "Guardar"** → el ViewModel arma el `CreateCustomerRequest` y llama a `ICreateCustomerInputPort.ExecuteAsync(request)`
2. **El caso de uso valida el DTO** con el Validator Hub (precondiciones técnicas)
3. **Verifica la unicidad** → `_repository.FindByDocumentAsync(...)` y `FindByEmailAsync(...)` ejecutan un `SELECT` en SQL Server
4. **Construye la entidad** → `Customer.Create(...)` valida cada Value Object en el dominio
5. **Persiste** → `_repository.AddAsync(customer)` agrega el cliente al `DbContext` y `SaveChangesAsync()` ejecuta el `INSERT`
6. **Notifica el éxito** → `_outputPort.HandleSuccessAsync(response)`
7. **El Presenter recibe el response** → lo guarda como `OperationResult`
8. **El ViewModel lee el resultado** → `if (_presenter.Result.IsSuccess)` muestra el mensaje
9. **El usuario ve el éxito** → "Cliente registrado con éxito"

Lo importante: **en ningún momento se rompe la arquitectura**. El caso de uso no sabe de SQL Server, el ViewModel no sabe de Entity Framework y Domain no sabe que existe una base de datos. Todo pasa a través de interfaces, y solo la Parte E sabe con qué clases concretas se cumplen.

Parte H — Ejercicio: CU-CLI-002 (Actualizar Cliente)

Ahora es tu turno de completar **CU-CLI-002: Actualizar Cliente** usando lo que aprendiste con CU-CLI-001. Esta parte no trae el código resuelto.

Punto de partida

Consultá la especificación en `cu_cli_002_actualizar_cliente.html`. De este caso de uso ya existe casi todo:

- `UpdateCustomerUseCase`, los DTOs (`UpdateCustomerRequest` / `UpdateCustomerResponse`) y los puertos, de `guia-cu-cli-002-actualizar-cliente.html`.

- `Customer.Update(...)` en Domain, que modifica solo los cuatro campos editables.
- Los métodos `GetByIdAsync` y `UpdateAsync` del repositorio, que acabás de escribir en la Parte D. **Infrastructure no necesita ningún cambio.**

Tu tarea

Falta la pieza de InterfaceAdapter que recibe el resultado del caso de uso. Implementá:

1. `UpdateCustomerPresenter` en `src/InterfaceAdapter/Customers/Presenters/`, que implemente `IUpdateCustomerOutputPort` reutilizando `BasePresenter<UpdateCustomerResponse>` (los tres canales base ya vienen resueltos; solo escribís los dos canales propios de este caso de uso). Exponé una propiedad `Response` como hace `CreateCustomerPresenter`.
2. **El registro en DI:** un método `AddUpdateCustomerPresenter` en el `DependencyContainer` de InterfaceAdapter que registre el presenter y su puerto de salida (de modo que ambos se resuelvan a la *misma* instancia).
3. **La comprobación contra la base:** registrá un cliente con CU-CLI-001, actualizalo con CU-CLI-002 y verificá en la tabla `Customers` los resultados de la tabla de abajo.

Cómo sabés que está bien

Situación	Resultado esperado
Actualizar email, teléfono, dirección y fecha de nacimiento de un cliente existente	`Result.IsSuccess` es verdadero y `Response` trae los valores confirmados. En la tabla cambian esas cuatro columnas y `UpdatedAt`; el nombre, el apellido y el documento quedan igual.
El `Id` no corresponde a ningún cliente	`IsSuccess` es falso, el mensaje incluye el `Id` y la tabla no cambia.
El nuevo email ya pertenece a *otro* cliente	`IsSuccess` es falso, el mensaje nombra al email y la fila conserva el email anterior.
Un dato que el dominio rechaza (por ejemplo una fecha de nacimiento de menor de edad)	`IsSuccess` es falso con el mensaje del dominio y la fila no cambia.

> **Consejo:** `CreateCustomerPresenter` es tu modelo. La diferencia es que acá los dos canales propios son "el cliente no existe" y "el email ya está registrado", en vez de documento y email duplicados.

> **No necesitás crear un ViewModel ni una View nueva.** En esta primera iteración alcanza con comprobar el caso de uso contra la base; las pantallas son parte de tus decisiones de diseño cuando exista el host.

Checklist de verificación

Archivo / Tarea	Verificación
`CommerX.Infrastructure.csproj`	Referencia a `Microsoft.EntityFrameworkCore.SqlServer` y `Microsoft.EntityFrameworkCore.Design`, ambos en 9.0.20; la solución compila sin errores ni advertencias
`Customer.cs`	Constructor `private` sin parámetros que inicializa los siete Value Objects con `null!`; `Create()` sigue siendo la única forma de crear un cliente desde el código de negocio
`CustomerConfiguration.cs`	Implementa `IEntityTypeConfiguration<Customer>`: tabla `Customers`, clave `Id`, `CreatedAt` y `UpdatedAt` mapeadas, siete `OwnsOne` con `Property(x => x.Value)` y su longitud máxima, índices únicos dentro del `OwnsOne` de `Document` y de `Email`, navegaciones requeridas
`CommerXDbContext.cs`	`DbSet<Customer>` y `OnModelCreating` con una sola línea `ApplyConfiguration(new CustomerConfiguration())`; no contiene detalles de mapeo
`CustomerRepository.cs`	Implementa los cinco métodos de `ICustomerRepository` con su firma exacta; compara por `.Value`; `AddAsync` y `UpdateAsync` terminan en `SaveChangesAsync`
`DependencyContainer.cs` (Infrastructure)	`AddInfrastructure(connectionString)` registra el `DbContext` con SQL Server y `ICustomerRepository` como Scoped; la cadena llega por parámetro
`CommerXDbContextFactory.cs` y `Migrations/`	`dotnet ef migrations add InitialCreate` genera la migración y `dotnet ef database update` crea la base `CommerX` con la tabla `Customers` (10 columnas, 3 índices)
`UpdateCustomerPresenter.cs` y `DependencyContainer.cs` (InterfaceAdapter)	El presenter hereda de `BasePresenter`, implementa los dos canales propios y el registro devuelve la misma instancia como presenter y como puerto de salida

Archivo / Tarea	Verificación
Comprobación contra la base	Un cliente registrado aparece en la tabla; un segundo con el mismo documento o email es rechazado; una actualización cambia solo los cuatro campos editables

Cuando registres un cliente, tiene que aparecer en la tabla `Customers`.
Podés verificarlo en el Explorador de objetos de SQL Server de Visual Studio o con Azure Data Studio.

Preguntas de reflexión

1. **¿Qué pasaría si olvidaras `await _db.SaveChangesAsync()` en `AddAsync()`?**
   Pista: el cliente se agregaría en memoria, pero ¿se guardaría en la base?
2. **¿Por qué usamos `OwnsOne()` en lugar de una tabla separada para `FirstName`?**
   Pista: `FirstName` es un Value Object pequeño, sin identidad propia. ¿Tiene sentido una tabla con su propia clave?
3. **Si mañana la aplicación cambia de SQL Server a PostgreSQL, ¿qué archivos cambian y cuáles no?**
   Pista: ¿el caso de uso? ¿el repositorio? ¿el `DependencyContainer`? ¿las migraciones? Pensá además en la búsqueda por email: `Email.Create` guarda el email en minúsculas, y SQL Server compara sin distinguir mayúsculas pero PostgreSQL sí las distingue.
4. **¿Por qué el caso de uso recibe `ICustomerRepository` y no `CustomerRepository`?**
   Pista: dirección de las dependencias. Si recibiera la clase concreta, ¿qué proyecto tendría que referenciar Application, y qué pasaría con las pruebas del caso de uso?
5. **¿Qué ventaja tiene que el mapeo de `Customer` esté en su propia clase de configuración y no dentro de `OnModelCreating`?**
   Pista: imaginá que el sistema tiene veinte entidades. ¿Qué archivo habría que abrir para cambiar la longitud de un campo? ¿Y qué pasaría con ese archivo si cambiara el motor de base de datos?
6. **¿Por qué `Customer` necesita un constructor sin parámetros, si el negocio nunca lo usa?**
   Pista: ¿quién crea el objeto cuando EF Core lee una fila? ¿Por qué no puede usar el constructor con los Value Objects?

7. **¿Por qué `CreatedAt` necesita una línea explícita en el `DbContext` y `UpdatedAt` casi no la necesitaría?**
   Pista: compará cómo se declaran las dos propiedades en `BaseEntity` (qué accesores tiene cada una) y qué mapea EF Core por convención. ¿Qué otra propiedad de solo lectura aparece en cada Value Object, y qué línea del `OwnsOne` la resuelve?